AI Curriculum Tutor

Project Alpha Prototype Report

Ron Wright

Team ZJL

Zachary Mullin, Jesse Boakye, Luis Zarate

12/1/2025

Typed single-spaced.

Typed with black text.

Typed with #11 font size.

Typed using Arial font.

Typed with one inch margins on sides, top and bottom.

(Please erase this page in your final document.)

Table of Contents

I. Introduction 1
II. Team Members – Bios and Project Roles 2

III. System Requirements Specification 3
- III.1. Use Cases 3
- III.2. Functional Requirements 4
  - III.2.1. [Module/Component Name] 4
  - III.2.2. [Next Module/Component Name] 5
- III.3. Non-Functional Requirements 6
- III.4. User Stories and Traceability Matrix 7
- III.5. Standards 8

IV. System Evolution 9

V. Architecture Design 10
- V.1. Overview 10
- V.2. Subsystem Decomposition 11
  - V.2.1. [Subsystem Name] 12
    - a) Description 12
    - b) Concepts and Algorithms Generated 13
    - c) Interface Description 13
  - V.2.2. [Next Subsystem Name] 14

VI. Data Design 15

VII. User Interface Design 16

VIII. Constraints 17

IX. Project Testing and Acceptance Plan 18
- IX.1. Overview 18
  - IX.1.1. Test Objectives and Schedule 19
  - IX.1.2. Scope 19
- IX.2. Testing Strategy 20
- IX.3. Test Plans 21
  - IX.3.1. Unit Testing 21
  - IX.3.2. Integration Testing 22
  - IX.3.3. System Testing 23
    - IX.3.3.1. Functional Testing 24
    - IX.3.3.2. Performance Testing 25
    - IX.3.3.3. Standards and Constraints Verification / Testing 26
    - IX.3.3.4. User Acceptance Testing 27
- IX.4. Environment Requirements 28

X. Alpha Prototype Description 29
X.1. [Subsystem Name] 30
X.1.1. Functions and Interfaces Implemented 30
X.1.2. Preliminary Tests 31
X.2. [Next Subsystem Name] 32

XI. Alpha Prototype Demonstration 33

XII. Future Work 34

Glossary 35

References 36

Appendices 37
Appendix A – User Stories and Acceptance Scenario
Appendix B – Example Testing Strategy
Appendix C – Additional Screenshots, Diagrams, and Test Evidence
Appendix n – As Needed

NOTE: COPY ALL THESE SECTIONS FROM PREVIOUS REPORT DELIVERABLES ASSIGNMENTS

I. Introduction 1
II. Team Members – Bios and Project Roles 2

III. System Requirements Specification 3
- III.1. Use Cases 3
- III.2. Functional Requirements 4
  - III.2.1. [Module/Component Name] 4
  - III.2.2. [Next Module/Component Name] 5
- III.3. Non-Functional Requirements 6
- III.4. User Stories and Traceability Matrix 7
- III.5. Standards 8

IV. System Evolution 9

V. Architecture Design 10
- V.1. Overview 10
- V.2. Subsystem Decomposition 11
  - V.2.1. [Subsystem Name] 12
    - a) Description 12
    - b) Concepts and Algorithms Generated 13
    - c) Interface Description 13
  - V.2.2. [Next Subsystem Name] 14

VI. Data Design 15

VII. User Interface Design 16

VIII. Constraints 17

IX. Project Testing and Acceptance Plan 18
- IX.1. Overview 18
  - IX.1.1. Test Objectives and Schedule 19
  - IX.1.2. Scope 19
- IX.2. Testing Strategy 20
- IX.3. Test Plans 21
  - IX.3.1. Unit Testing 21
  - IX.3.2. Integration Testing 22
  - IX.3.3. System Testing 23
    - IX.3.3.1. Functional Testing 24
    - IX.3.3.2. Performance Testing 25
    - IX.3.3.3. Standards and Constraints Verification / Testing 26
    - IX.3.3.4. User Acceptance Testing 27
- IX.4. Environment Requirements 28

I. Introduction

Educational technologies have historically faced resistance before being embraced as essential classroom tools When calculators were first introduced, many feared they would erode students' ability to think mathematically. Smartphones similarly were stigmatized in schools before becoming indispensable for learning and communication. Artificial intelligence is sparking a similar debate as its capability is far greater than the tools before it. In a global context where AI is advancing rapidly, the U.S. education system cannot afford to fall behind.

This project is motivated by the need to explore safe, practical, and student-centered ways of incorporating AI tutors into classrooms. The goal is not to compete with commercial systems but rather to understand what is already available online, test smaller models that can be run locally, and develop curricular frameworks that allow both teachers and students to create and adapt personal AI tutors. The broader vision is to move beyond the one-size-fits-all "factory" model of schooling and toward a personalized learning environment where AI supports students in building knowledge at their own pace under teacher guidance.

By conducting evaluations of lightweight language models, prototyping local deployments, and producing teaching guidelines and legislative recommendations, the project seeks to demonstrate how AI tutors can be integrated responsibly. The outcome will not only provide practical examples of student-built tutors but also highlight pathways for administrators and policymakers to support safe, equitable, and scalable AI adoption in education.

II. Team Members - Bios and Project Roles

Luis Zarate (luis.zarate@wsu.edu)

- Major: Computer Science
- Hometown: Issaquah, WA
- Education:
    - Senior, Computer Science at Washington State University
- Skills:
    - Languages: Python, JavaScript, C++, HTML, CSS, SQL
    - Frameworks/Tools: React, Node.js, Git/GitHub
    - Areas: Web Development, AI/ML Integration, UI/UX Design, Project Management
- Projects:
    - EventHub (campus event discovery app)
    - CougarConnect (student networking platform)
    - AlumniNetwork (alumni-student connection site)
    - Rust Forge (Rust tutorial project)
    - PostScript Interpreter (Python-based language project)
- Other Interests:
    - Education technology & accessibility
    - Community building through software
    - Hiking and photography

Zachary Mullin (zachary.mullin@wsu.edu**)**

Hometown: Bonney Lake, WA

Education:

Senior in Computer Science, B.S. at Washington State University

Technical Skills:

- Programming Languages: C++, C, C#, Java, Python
- Frameworks and Tools: .NET, SFML, PyTorch, TensorFlow, Hadoop, Apache Spark
- Areas of Interest: AI/ML Development, Frontend Development, Cybersecurity

Past Projects/Initiatives:

- **YouTube Analytics -** Application that analyzes YouTube video rankings and recommends related videos using Hadoop, Apache Spark and Pandas to process big data operations.
- **Location-based Messenger -** Chat application using HTML templates and the Django web framework that connects users to an online chat room for their city, using their location to accurately connect them to the right chat room.
- **Discord Activities Planner -** Discord bot application written in Python that collects nominations for activities from users and runs a poll once a week to determine the current weekend's activity.

Jesse Boakye jesse.boakye@wsu.edu

linkedin.com/in/jesse-boakye

https://github.com/jesseboakye

EDUCATION

Bachelor of Science in Computer Science Expected May 2026

Washington State University, Pullman, WA
Relevant Coursework: Computer Architecture, Advanced Data Structures, Software Engineering

SKILLS

Languages: Java, Python, C++, JavaScript, HTML/CSS, SQL
Frameworks/Tools: React, Node.js, Express.js, Redux Toolkit, Firebase, JWT, Google OAuth
Databases: MongoDB, MySQL
Systems/other: Windows, Linux, Git, REST APIs, Agile

PROJECTS

Modern Real Estate Marketplace, Personal project 2024

- Developed a full-stack real estate marketplace using MERN stack.
- Implemented secure authentication with JWT, Firebase, and Google OAuth reducing unauthorized access attempts by 100%.
- Built an advanced search functionality with Redux Toolkit, improving user query speed by 30%.
- Designed MongoDB schema enabling full CRUD operations and integrated image upload system for property management.

Restaurant Automation, Group project 2023

- Engineered an app to coordinate restaurant staff, enabling real-time table management and order tracking.
- Automated data collection and processing, providing managers with instant revenue analytics and trend visualizations.
- Optimized backend processes, reducing data retrieval latency by 40%.

Educational Resource Database, UW class project
2021

- Designed and implemented sorting algorithms (quicksort, merge sort) to improve retrieval time of educational resources.
- Reduced average search time by 50% through algorithm optimization and data indexing.

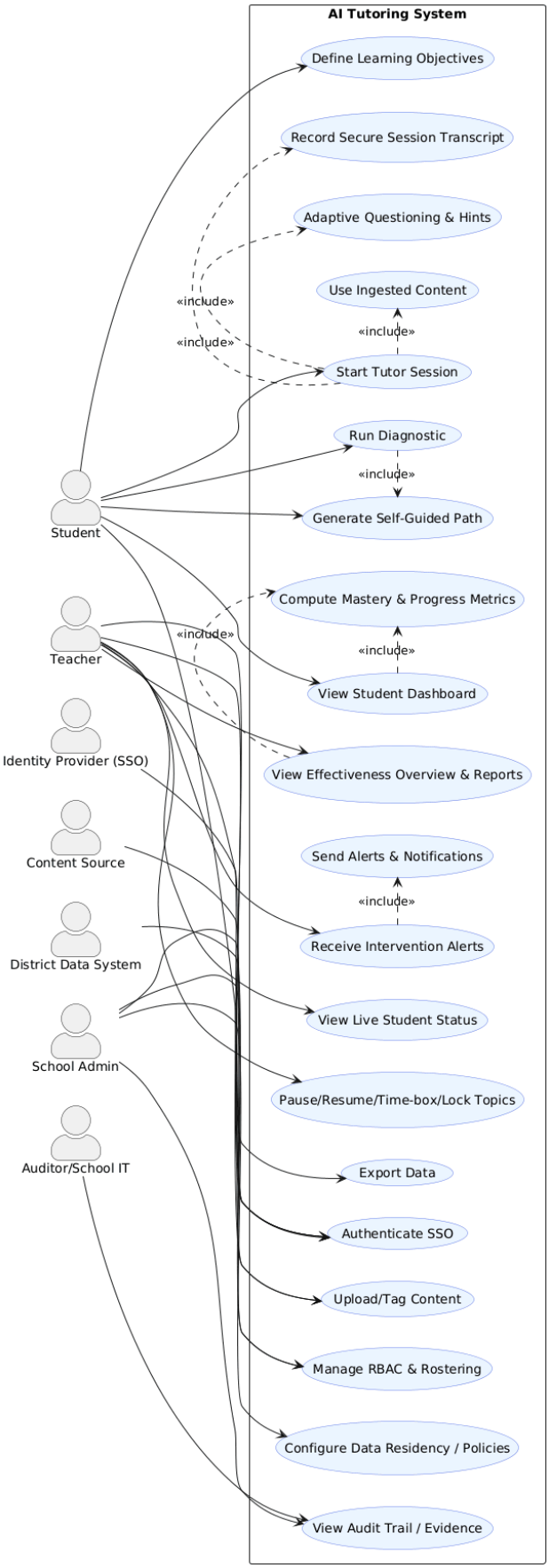
Project Title

III. System Requirements Specification

This section outlines the key functional and non-functional requirements, as well as the use cases, for the AI tutor system. The functional requirements capture the core capabilities, including role-based access, adaptive tutoring, content ingestion, mastery tracking, diagnostics, progress dashboards, classroom management, interventions, and evidence logging. The non-functional requirements specify the system quality attributes. The use case provides scenarios that highlight how students, teachers, and administrators will interact with the system in practice.

III.1. Use Cases

The following consolidated use case illustrates how the AI Tutor system supports its three primary user roles: Students, Teachers, and School Admins. Each actor interacts with the system under different circumstances, as shown in the UML diagram below.



Project Title

III.2 Functional Requirements

III.2.1. FR-1 Role-based Access & Rostering

Role-based Access & Rostering: The system shall support Student, Teacher, and School Admin roles, ensuring each has least-privilege permissions aligned to their responsibilities.
Source: Client requirement for differentiated system access and security.
Priority: Level 0 (Essential and required functionality)

III.2.2. FR-2 Adaptive Tutor Sessions

Adaptive Tutor Sessions: The system shall support one-on-one tutor sessions that adapt question difficulty based on recent student responses and provide hints and step-by-step feedback.
Source: Client vision for personalized tutoring. *(Flagged as difficult to implement in meeting.)*
Priority: Level 2 (Extra feature / stretch goal)

III.2.3. FR-3 Content Ingestion & Tagging

Content Ingestion & Tagging: The system will support PDFs, links, and other learning materials via a RAG LLM. The tutor uses this content in sessions, and materials may be tagged by topics or standards.
Source: Client request for flexibility in learning content.
Priority: Level 0 (Essential and required functionality)

III.2.4. FR-4 Student-defined Learning Objectives & Mastery Tracking

Student-defined Learning Objectives & Mastery Tracking: The system tracks mastery against thresholds (e.g., ≥80% over recent attempts) and adapts practice according to student-defined objectives or topics of study.
Source: Client emphasis on student-driven learning, not teacher-enforced curriculum.
Priority: Level 0 (Essential and required functionality)

III.2.5. FR-5 Diagnostic & Self-guided Learning Paths

Diagnostic & Self-guided Learning Paths: The system shall provide diagnostics for a chosen topic/unit that generate a personalized learning path. Teachers may view summaries but do not prescribe paths.
Source: Client requirement for un-traditional, student-controlled progression with teacher as monitor.
Priority: Level 0 (Essential and required functionality)

III.2.6. FR-6 Progress Dashboards (Student-led, Teacher Overview)

Progress Dashboards (Student-led, Teacher Overview): The system will provide progress dashboards for all roles. Student dashboard shows mastery by topic, streaks, goals, time-on-task, and suggested next steps. Teacher/Admin dashboard provides overview of tutoring effectiveness, student growth over time, and highlights struggling students. Dashboards support data exports (CSV/other file types).
Source: Client request for student empowerment with teacher monitoring role.
Priority: Level 0 (Essential and required functionality); data exporting is Level 1 (Desirable functionality)

III.2.7. FR-7 Classroom Monitoring & Live Status

Classroom Monitoring & Live Status: The system will have secure monitoring tools for teachers and admins. Teachers shall have tools to view live student status, pause/resume tutoring, lock topics, and time-box sessions as needed. Controls are designed for monitoring and classroom flow, not rigid enforcement.
Source: Idea by the team that was agreed upon by the client.
Priority: Level 1 (Desirable functionality)

III.2.8. FR-8 Intervention & Notifications

Intervention & Notifications: The system shall alert teachers when a student is struggling to progress even with tutor assistance, enabling timely teacher intervention.
Source: Client and team agreed-upon idea from meeting.
Priority: Level 1 (Desirable functionality)

III.2.9. FR-9 Evidence & Audit Trail

Evidence & Audit Trail: The system shall store session transcripts (student prompts, tutor responses, hints shown) with timestamps for security, review, and auditing.
Source: Education security requirement that was noted by the client.
Priority: Level 0 (Essential and required functionality)

III.3. Non-Functional Requirements

Privacy & compliance:
System shall comply with FERPA and COPPA; data residency per district contract.

Security:
System will be secured with SSO via SAML/OAuth2; RBAC; encryption in transit (TLS 1.2+) and at rest (AES-256).

Performance:
System will have a median tutor response time ≤ 2.0 s; 95th percentile ≤ 5.0 s during school hours (8am–3pm local)
*These are goals set by the team, but we aren't sure yet if they are fully achievable with the hardware we are working with.

Availability & scalability:
System will have a service uptime ≥ 99.9% monthly.

Accessibility & inclusivity:
System will conform to WCAG 2.2 AA; supports screen readers, keyboard-only nav, captions for audio.

Usability:
System will be documented well enough to allow new teachers to complete core tasks (create class, assign session, view report) with ≤ 10 minutes of guided onboarding.

Maintainability & observability:
System will be upgradable over time with new language models because of the rapidly advancing field of LLMs.

III.4. Standards

Our system adheres to several established international and industry standards to ensure accessibility, privacy, ethical integrity, and documentation quality throughout the development process.

1. **IEEE 830-1998 – Software Requirements Specification (SRS):**
   All functional and non-functional requirements were documented following the IEEE 830 structure, which provides a consistent framework for clarity and traceability. Each requirement in our report is uniquely identified (e.g., FR-1, NFR-3) and tied to its intended verification activity. This ensures completeness across all project phases and supports ongoing maintainability as the system evolves.

2. **IEEE 1016-2009 – Software Design Description (SDD):**
   To maintain structured design documentation, IEEE 1016 guidelines were followed when describing components, data flow, and subsystem interactions. These standards informed our diagrams and subsystem breakdowns such as the LLM Implementation and Dashboard Framework allowing for clear communication between developers and reviewers.

3. **W3C WCAG 2.2 (Level AA) – Web Content Accessibility Guidelines:**
   Because our platform will be used by students of varying abilities, accessibility is a core design principle. WCAG 2.2 Level AA criteria guided the development of the user interface, including high-contrast text ratios (≥4.5:1), keyboard navigation, and screen reader compatibility. These standards ensure that every student can interact effectively with the AI tutor, regardless of ability or device.

4. **FERPA (Family Educational Rights and Privacy Act) and COPPA (Children's Online Privacy Protection Act):**
   Given the educational context, student privacy and data protection are essential. FERPA compliance ensures that student progress data is securely stored and only accessible to authorized mentors or administrators. COPPA guidelines further ensure that no personal information is collected from minors without verified consent, maintaining ethical alignment with district policies and legal standards.

5. **AES-256 (NIST FIPS 197) – Encryption Standard:**
   All sensitive data including user profiles, quiz results, and chat transcripts are encrypted at rest using AES-256 and transmitted over HTTPS with TLS 1.3. This combination ensures strong end-to-end protection against unauthorized access or data breaches within school district systems.

6. **IEEE 7000-2021 – Ethical Concerns During System Design:**
   This standard guided the integration of ethical considerations during early design phases. In particular, the system's AI components were designed with transparency and fairness in mind, preventing the tutor from providing direct answers or biased explanations. This ensures that AI behavior aligns with human educational values and promotes learning integrity.

Together, these standards establish the foundation for a safe, compliant, and equitable AI tutoring environment. They ensure that our system not only performs effectively but also upholds the professional, legal, and ethical expectations of educational software used in public institutions.

IV. System Evolution

Our project is based on the assumption that lightweight AI models, such as an 8B parameter LLM, can be deployed in classroom settings with acceptable performance for multiple simultaneous users. Current discussions suggest that performance may degrade after 20–32 students, so our design anticipates both hardware limitations and the need for scalability. As computing hardware evolves, larger and more efficient models will become accessible locally, which would expand the capabilities of the tutors without requiring a major redesign.

User needs are expected to evolve as well. At launch, students will primarily seek simple, personalized tutors that adapt to their individual learning goals, while teachers will want tools to monitor progress and receive alerts when intervention is needed. Over time, demands may grow for accessibility features such as audio interfaces, as well as for broader functionality like multilingual support or integration with other classroom platforms. Our system will be designed

modularly so that these features can be introduced without disrupting the core tutor–student interaction.

Risk points include hardware scalability, where insufficient resources could restrict adoption in larger classrooms, and policy concerns, since schools may set limits on AI use in instruction. Security also presents a major consideration: protecting student data and ensuring responsible AI usage will remain priorities. By identifying these risks early, we can mitigate them through design decisions such as cloud-assisted fallback options, lightweight deployment strategies, and strong privacy protections.

Ultimately, the system is expected to evolve alongside advances in AI and changing educational practices. By building a flexible framework that supports both student-driven personalization and teacher oversight, the project will remain adaptable and relevant as classroom needs and technology continue to advance.

V. Architecture Design

V.1. Overview

The AI Tutoring System follows a layered and service-oriented architecture that separates user interaction, application logic, data management, and AI integration into independent yet connected subsystems. This structure offers a clear top-level design view and provides a foundation for the detailed subsystem design that follows. The system follows a four-layered architecture (Presentation, Application, Integration, and Data).Each layer encapsulates distinct responsibilities and communicates through defined APIs. This layered structure improves modularity, security, and scalability.

System Responsibilities and Communication Flow

User requests flow downward through the layers:
Browser → Presentation → Application → Integration → LLM / Data, while results propagate upward with added metadata (progress updates, quiz scores, etc.).

For example:
A student submits a question → the API Gateway routes it to the Tutor Session Service → which uses the LLM Connector to query the Local LLM → the response is validated and returned to the student with mastery updates sent to the Progress Service.

Architectural Diagram:

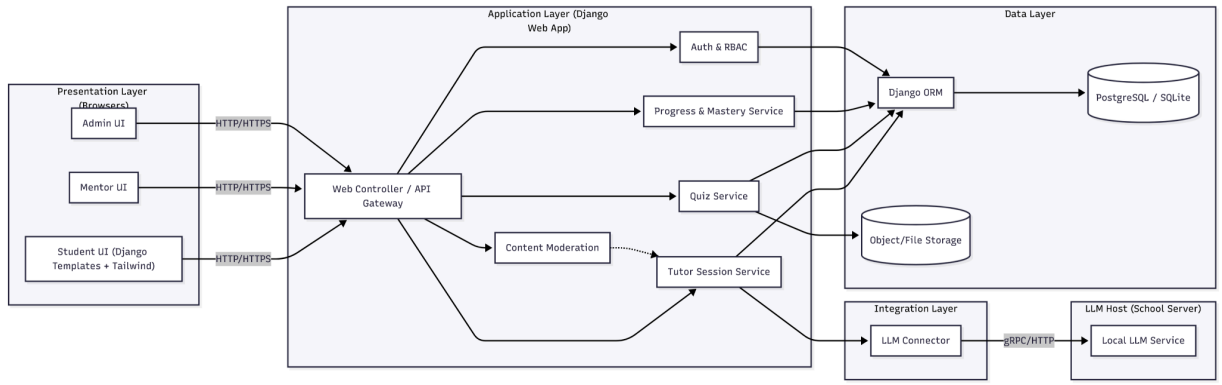


Figure III-1 — High-Level Component Diagram

V.2. Subsystem Decomposition

The AI Tutoring System was decomposed into several subsystems based on functional responsibilities, cohesion, and ease of independent development. Each subsystem encapsulates a specific area of functionality such as user interaction, learning logic, progress tracking, or AI communication, all while exposing clear interfaces to the others.
This modular approach promotes high cohesion within subsystems and low coupling between them, enabling parallel development, testability, and maintainability.

V.2.1 Decomposition Rationale

Building upon the layered architecture introduced in Section III.1, the system is further divided into specific subsystems that implement each layer's responsibilities.

1. **Presentation Layer -** Provides front-end interfaces for different user roles to interact with the system.

2. **Application Layer -** Contains the core business logic, API endpoints, and modular services that process requests and coordinate data flow.

3. **Data Layer -** Responsible for persistence and retrieval of structured and unstructured data.

4. **Integration Layer -** Connects the Django application to external AI/LLM systems for tutoring and content generation.

This decomposition aligns with the layered structure described in Section III.1, but at the subsystem level. Each major service (Tutor, Quiz, Progress, etc.) resides within one of these layers. This ensures high cohesion within each subsystem and low coupling between layers, allowing independent development and testing.

V.2.2 Subsystems and Responsibilities

Subsystem	Primary Responsibilities	Provided Interfaces	Required Interfaces
Web Controller / API Gateway	Routes HTTP requests, validates input, and forwards calls to the proper service.	REST endpoints for Tutor, Quiz, Progress, Auth.	Calls internal service APIs.
Auth & RBAC	Handles authentication, session tokens, and role-based access.	Authentication API.	Database Access.
Tutor Session Service	Manages active chat sessions, builds LLM prompts, parses responses, and updates mastery data.	Tutor API (responses + metadata).	LLM Connector, Moderation, Progress Service, ORM
Quiz Service	Generates, and grades quizzes created by the AI tutor.	Quiz API.	-
Content Moderation	Filters and flags AI responses for inappropriate or off-policy content.	Moderation API.	Tutor Service callback.
LLM Connector	Formats prompts and relays requests to the local LLM server.	LLM API.	HTTP/gRPC connection to LLM.
Progress & Mastery Service	Aggregates results and tracks learning progress for each student.	Progress API	-
ORM & Database Layer	Persists users, and sessions	Repository interface.	Database engine.

Illustrative Sequence

Figure III-B below demonstrates the “Ask AI Tutor for Help” use case. It shows how the Student UI, API Gateway, Tutor Session Service, Moderation, LLM Connector, Local LLM, and Progress Service interact to fulfill a tutoring request.

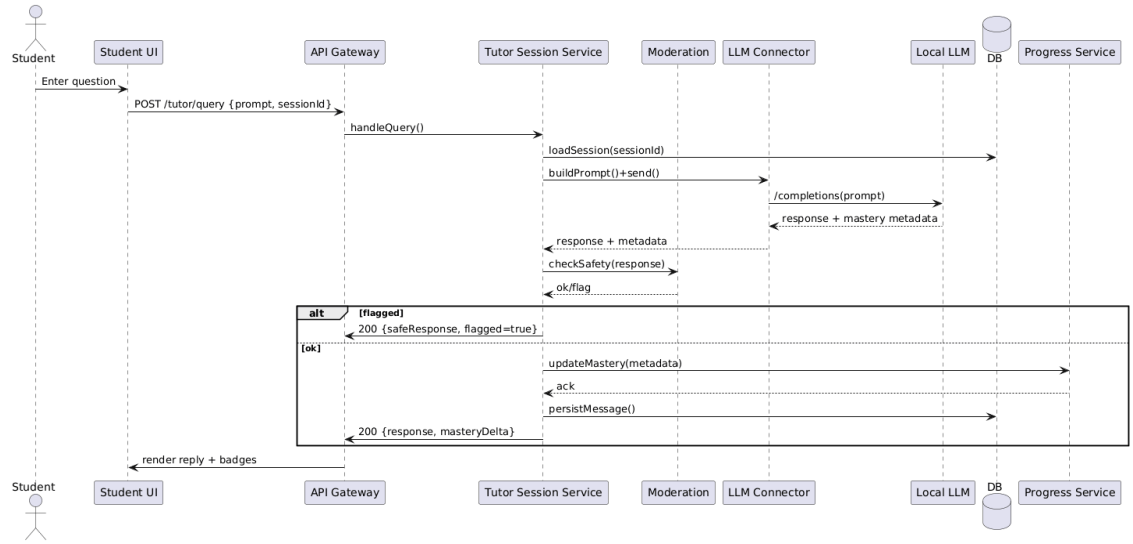


Figure III-B — Sequence: “Ask AI Tutor for Help”

V.3.1. LLM Implementation

a) Description

This system communicates with the large-language model and will both send prompts to the large language model as well as store important information the large-language model provides. This information usually pertains to student information and mastery tracking, which is then used by the system to update these mastery trackers on the student dashboard.

b) Concepts and Algorithms Generated

Initial Prompt - This prompt is specifically designed to make it impossible or very difficult to inject any further instructions into the LLM for security reasons. This also provides the rigid details for how the LLM is supposed to respond to students and their needs. It provides specific formatting instructions for the AI responses to ensure a clean UI appearance using delimiters while also providing metadata for the system, like quizzes, mastery tracking information, and more.

Mastery Tracking - The LLM will come up with mastery tracking metadata for the student based on the current session. This mastery tracking data will then be added to existing mastery tracking data for each session, which will be sent to the Mastery Tracking subsystem to be built upon.

c) Interface Description

Services Provided:

1. Mastery Tracking

Service provided to: Mastery Tracking

Description: The LLM will output metadata included with the response that informs the Mastery Tracking system on how to update the current data. For example, if a student successfully answers a certain amount of quiz questions on linear equations, then the mastery tracking for Algebra will be updated, showing that the student has learned a certain amount of foundational Algebra topics.

Services Required:

User Prompts - Provided by the frontend's user.

V.3.2. Dashboard Framework (Django)

a) Description

The dashboard will provide both an interface to communicate with the tutor for students, as well as mastery tracking systems for both students and mentors.

b) Concepts and Algorithms Generated

Send to LLM - The system will take an input of the user's prompt and send it to the LLM for a response.

Load Website - The system will display a user interface showing the user's past chat history and show a dashboard of the user's mastery of the topics they are currently studying.

Log In - The website will allow a student or mentor to log into the system, displaying different abilities and tools whether the account belongs to a student or a mentor.

c) Interface Description

Services Provided:

1. User Prompts

The system will read a prompt from the UI text box that the user inputs and send that to the LLM.

Services Required:

LLM Responses - The system will require the response provided by the LLM containing the tutor's response as well as any metadata, including a quiz or any mastery tracking data for updating the dashboard.

VI. Data Design

For this application, three primary data structures will be used to support the AI tutoring system's workflow: User, TutorSessions, and QuizRecord. Each structure plays a key role in managing user profiles, AI interaction, and assessment tracking. Together, these components form the foundation for a scalable and personalized tutoring environment.

User Structure: Each user (student, teacher, or administrator) is represented by a User object. This object contains the following fields.

- UserID: Unique identifier for each user.
- Role: Specifies user type (student, teacher, or admin) and determines permissions
- Name: The user's display name.
- Email: Used for authentication and notifications
- Preferences: A JSON object storing user-selected tutor settings such as subject and difficulty level

The UserID serves as a primary key and is referenced by other structures to ensure user-specific data integrity.

TutorSession Structure: the TutorSession object represents a single interactive tutoring session between a student and the AI model.

- SessionID: Unique identifier for each tutoring session.
- UserID: Foreign key linking to the User structure.
- Topic: The main subject or concept discussed in the session.
- ChatHistory: A structured JSON object recording the dialogue between the student and AI tutor.
- SessionStatus: Tracks whether a session is ongoing, paused, or completed.
- StartTime/EndTime: Timestamps recording the duration of the session.

This structure allows the system to log, resume, and review sessions as needed while maintaining a history of each student's learning interactions.

QuizRecord Structure: The QuizRecord data structure manages assessments created by the AI tutor and completed by the student.

- QuizID: Unique identifier for quiz.
- UserID: Foreign key linking the record to a specific user.
- SessionID: Optional link to the tutoring session where the quiz was created.
- QuizData: JSON object containing the quiz content generated by the tutor.

- StudentAnswers: Student-submitted responses in JSON or CSV format.
- GradedResults: JSON object storing the tutor's evaluation and feedback.
- Score: Numeric value representing the student's performance.

This design allows quizzes and evaluations to be stored, retrieved, and analyzed for both student progress and teacher review.

Database Design Overview

The system's database will likely use a relational schema (SQLite) to ensure strong consistency across user, session, and quiz data. Foreign key relationships (UserID -> TutorSession, UserID -> QuizRecord) enforce data integrity and simplify querying user progress and history. In future versions, additional structures will support RAG (Retrieval-Augmented Generation) content ingestion for curriculum materials, as well as TeacherAnalytics table for monitoring student performance trends and alerts.

VII. User Interface Design

Our AI Tutoring System currently features a functional web-based user interface focused on enabling direct interaction between the student and the AI tutor. The interface has been designed to be intuitive, minimalistic, and focused on usability, allowing students to easily communicate with the AI model and receive feedback in real time.

Current Design:

Upon launching the web application, users are presented with a chat-based interface. The main page is centered around a chat window that facilitates conversations between the student and the AI tutor. A simple message log displays both users inputs and AI-generated responses in an alternating format, providing an experience similar to a modern messaging platform.

At the bottom of the interface, there is a text input box where the user can type their question or request for help, accompanied by a "Send" button that submits the query to the local LLM. The chat updates dynamically with the AI's response, allowing students to maintain an ongoing, interactive tutoring session without page refreshes.

This current chat interface supports the core use case of "Ask AI tutor for Help", allowing students to engage with the system to learn new material, clarify questions, or request explanations on specific topics.

Planned Enhancements:

In upcoming development iterations, additional interface components will be integrated, including:

- Student Dashboard: Displays learning progress, mastery metrics, and upcoming goals.
- Mentor Dashboard: Allows mentors to monitor student progress, review session history, and identify students who may need assistance.
- User Authentication: A login and registration system to enable personalized tutoring sessions and progress tracking.
- Session History Panel: A collapsible sidebar for reviewing past tutoring interactions and resuming previous sessions.

These planned features will further support key use cases such as “Track Learning Progress”, “Review Previous Sessions”, and “Monitor Student Performance”.

Design Approach:

The interface emphasizes accessibility and simplicity, ensuring that students of varying technical backgrounds can use it easily. Future updates will maintain a consistent design language utilizing clean typography, intuitive layout, and responsive design principles while expanding functionality.

VIII. Constraints

1. Hardware and Performance Constraints:
Since the system uses a locally hosted large language model (LLM), performance is limited by the computing power available on the host machine. Running inference locally requires managing memory efficiently and optimizing response generation speed to ensure smooth interaction for users.

2. Storage Constraints:
We are currently using SQLite for data storage, which provides simplicity and portability but limits scalability for large datasets or concurrent access. This constraint influences how we manage session history and user data.

3. Time and Resource Constraints:
As a student project developed within one academic year, we must balance feature scope with available time. Our focus is on building a functional and reliable prototype rather than a fully production-ready system.

4. Model and Integration Constraints:
The chosen LLM must run locally, which restricts us from using cloud-based APIs or models that require external connections. This impacts both performance and the variety of available model features.

5. Usability and Design Constraints:
The current UI is intentionally minimal, focusing on core tutoring functionality (chat interface). Future iterations will expand to include analytics and user management, but current scope limits UI complexity.

Throughout the design process, we've had to balance privacy, performance, and scalability. Running the LLM locally ensures stronger data privacy and offline availability but comes at the cost of slower processing and limited model size compared to cloud-based alternatives. Similarly, using SQLite simplifies deployment and development but restricts multi-user concurrency and long-term scalability. These trade-offs were made intentionally to prioritize system reliability, security, and feasibility within the project's timeline and available resources.

IX. Testing and Acceptance Plans

IX.1. Project Overview

This section provides an overview of the testing and acceptance methodology for the AI Tutor application. The goal of testing is to ensure that the system functions reliably and supports the learning experience for both students and teachers. The AI Tutor platform includes several major features that require verification, including user authentication, role-based access control, classroom and roster management, automatic student account generation from CSV files, adaptive assessment and quiz delivery, classroom monitoring, intervention alerts for teachers, and a retrieval-augmented generation model for personalized instruction. In addition, both the teacher and student-facing web interfaces will be evaluated to ensure that users can navigate the system effectively and access the features required for day-to-day use.

IX.1.2. Test Objectives and Schedule

The objective of this test plan is to confirm that the AI Tutor application satisfies both its functional and non-functional requirements and that its components operate consistently under realistic usage conditions. Testing will focus on validating the core subsystems, including user management, student performance tracking, adaptive question logic, and the interaction between the web interface, database, and language model processing pipeline. Testing will evaluate each feature individually, as well as the integration of these components within the complete system.

Testing resources will include the application source code, test documentation, and appropriate Python testing frameworks. The testing schedule begins with requirement-based validation, followed by automated unit and integration testing, manual system testing, performance evaluation where relevant, and final user acceptance testing prior to deployment. These activities will produce documented test results, updated artifacts, and evidence of successful execution, ensuring that the application is ready for operational use.

IX.1.3. Scope

This section defines the planning, approach, and resources for testing the AI Tutor application. It outlines the testing strategies and processes that will be used to evaluate the system, including unit, integration, and system-level testing. The scope includes both backend and frontend components, as well as requirements related to deployment environments, supporting software, and test tools. The goal of this plan is to ensure that the AI Tutor application meets the expected standards of functionality, reliability, and usability before release.

IX.2. Testing Strategy

The testing strategy for the AI Tutor system follows a requirement-driven and iterative verification process designed to ensure that all functional and non-functional requirements are validated against realistic classroom scenarios. All test cases will be derived directly from the Software Requirements Specification (SRS), which includes the role-based access control, adaptive tutoring logic, content ingestion workflows, mastery tracking, dashboard analytics, intervention notifications, and evidence logging. This approach ensures a full coverage of essential learning, monitoring and administrative capabilities which is essential for school districts.

Testing will follow a structured flow beginning with requirement analysis and traceability mapping. We will first confirm that each functional and non-functional requirement has an associated test objective. Unit testing will be performed on isolated components such as authentication modules, CSV-based roster ingestion, quiz logic functions, and the RAG retrieval pipeline. Once the individual modules pass unit testing, integration testing will focus on validating the interactions between subsystems– such as how the web interface communicates with the Django backend, the MySQL database, and the language model processing layer. After integration, we will perform a full system testing to evaluate the end to end behavior of the platform under real usage conditions to ensure it complies with our scenarios in the SRS and use cases. The final phase of the process will involve user acceptance testing with teachers and students to validate functionality against our sponsor expectations.

The project will use **Continuous Integration** (CI) via Github actions to automate unit and integration tests on every pull request. Because the system has multiple interacting services (Django backend, database, LLM interface, content ingestion pipeline), CI ensures that the components continue to function correctly as features evolve. We will not be using Continuous Delivery because the future deployment on school servers will require manual security review and administrative approval.

This testing strategy is aligned with the project's constraints, system standards, and architectural responsibilities. Because the system handles sensitive student data, incorporates accessibility requirements, and relies on a performance-bound LLM subsystem, the strategy emphasizes repeatable testing, traceability, measurable verification, and early defect detection.

IX.3. Test Plans

Below are the different kinds of testing we will use and their associated test plans.

IX.3.1. Unit Testing

Due to the nondeterministic nature of the LLM that we use for a large portion of this software, it is very difficult to unit-test this portion of the system. Instead, we will only be creating unit tests for parts of the program that don't require use of the LLM. This includes but is not fully limited to:

- Account creation
- Bulk (CSV) account creation
- Logging into the website
- Loading profile
- Loading mastery tracking data
- Sending intervention alerts to a teacher account
- Saving and loading student-defined objectives
- Testing of the Model Swapper helper program

IX.3.2. Integration Testing

Similar to unit testing, it is difficult to run integration tests on the LLM, because it is unpredictable and nondeterministic, meaning that we cannot assert its answers against a predicted outcome. Instead, we will once again focus on the areas of the system that do not use the LLM and run integration tests on that.

The primary system that will use integration testing is the account creation and logging in system. We will test account creation and make sure that an account can be created or bulk-created with CSVs and then immediately be logged into, as well as making sure the user can do every appropriate account feature such as changing their password.

IX.3. System Testing

System testing is a type of black box testing that tests all the components together, seen as a single system to identify faults with respect to the scenarios from the overall requirements specifications. Entire system is tested as per the requirements.

During system testing, several activities are performed:

- Setting up server application
- Choosing model using the Model Swapper
- Creating a teacher or admin account
- Bulk-creating student accounts using a CSV
- Logging into a student account
- Running the LLM tutor and asking it questions
- Asking for a quiz
- Verifying that mastery tracking reports are updated based on conversation
- Viewing mastery tracking reports on teacher/admin account

IX.3.1. Functional testing:

Test of functional requirements (from requirements specification). The goal is to select those tests that are relevant to the user and have a high probability of uncovering failure.

Functional testing will be the main source of testing for the LLM. We will give it standardized prompts, such as the ones below, covering a variety of topics and will draw from RAG PDFs:

- “Can you tell me about y = mx + b?”
- “Who were the 5 good emperors of Rome?”
- “What are the planets in the solar system?”
- “Can you summarize my solar system notes?”

We will also ask it for prompts that give quizzes to test the quiz system.

- “Can you give me a quiz on y = mx + b with 2 questions?”
- “Can you give me a quiz on the 5 good emperors?”
- “Can you give me a quiz based on my notes?”

The answers to these prompts will have to be human-verified because of the nondeterministic nature of the LLM. As such, we can’t write unit tests for the LLM. The acceptance criteria for the question prompts will be that the LLM answers the question provided in a detailed and easy-to-understand way.

The quiz system’s acceptance criteria will be that it generates the quiz JSON successfully, and the questions and answers it generates are relevant and correct to the quiz that was asked for. For example, if we ask for a quiz on y = mx + b and it gives us a quiz on the 5 good emperors of Rome, or if we ask for a quiz on the 5 good emperors of Rome and it gives “Nero” as the correct answer for “Which of these emperors is one of the 5 good emperors?”, then we will know there is something wrong with the LLM’s information or something wrong with the info provided in the RAG.

IX.3.2. Performance testing:

Performance tests check whether the nonfunctional requirements and additional design goals from the design document are satisfied. In stress testing, the system is stressed beyond its specifications to check how and when it fails.

We will time the LLM’s performance on a variety of machines, and we will take into consideration the requirements of the currently used model. If the model is supposed to run well on a very powerful server with 64 GB of RAM and 18GB of VRAM, and yet it is taking several minutes to run, we will know that something is wrong. The acceptance criteria will be variable for these kinds of tests, but we will try and stick close to our initial non-functional requirements for grading the performance.

IX.3.3. Standards and Constraints Verification

Verify and/or test standards from ‘Requirements’ section and constraints from ‘Solution Approach’ section. Verification / tests can be manual or via automated tools.

Our biggest constraint that will need testing is our hardware constraints. Due to using a large language model as the brain of the entire system, it will likely need a powerful system with lots of memory, but we will test and optimize to see if it can run on systems that are less powerful.

We also need to test the constraints of our UI. What can the user do/not do? Who has permissions to do what, and are they able to see what others can do or is it invisible to them? We need to make sure that students cannot perform actions that teachers are able to, or that they are able to access content that they aren’t supposed to have access to. Standards such as FERPA, COPPA, and WCAG 2.2 AA will be verified through code audit, UI accessibility scans, and review of data-handling workflows.

IX.3.4. User Acceptance Testing:

Acceptance testing and installation testing check the system against the project agreement. The purpose is to confirm that the system is ready for operational use. During acceptance test, end-users (customers) of the system compare the system to its initial requirements (if necessary) with help by the developers.

We will be running the program on the client’s school district server and have students picked by the client to test it. This will also play into our performance tests, as we will be able to stress-test the system and make sure it can handle several people asking questions from the AI at once.

IX.4. Environment Requirements

The testing environment for the AI Tutor system consists primarily of standard developer machines capable of running modern Python (3.10+) and Django 5.x. No special hardware is required; any workstation with a stable operating system (Windows, macOS, or Linux) and moderate resources can support local testing. Local tests rely on Django’s built-in tools, the SQLite development database, and optional CSV ingestion for bulk student creation.

For integration and system-level testing, a consistent environment will be provided through containerized setups such as Docker or a Linux-based CI runner. This environment will include all required dependencies Django, language model interfaces, and CSV processing utilities to ensure reproducible test results across the team. Internet access may be required for components that interact with external LLM APIs or remote resources.

If used, CI tools such as GitHub Actions will automate unit and integration tests within an isolated container, ensuring all branches pass testing before being merged. This setup allows the team to maintain reliability and consistency across development, testing, and deployment.

X. Alpha Prototype Description

X.I. Web Service

The system runs on the Django Web Framework in Python 3. The system is intended to run on a server affiliated with the school district, but for the prototype, is intended to run on a localhost or on a LAN connection.

To activate the server, you run “python manage.py runserver” in the command line in the folder where you download the program.

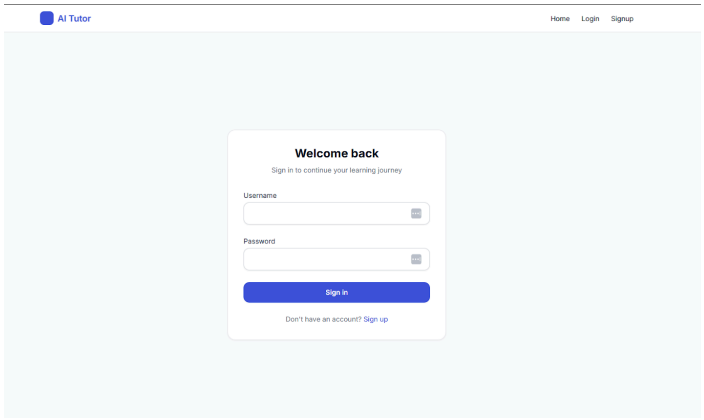
X.II. Auth and RBAC

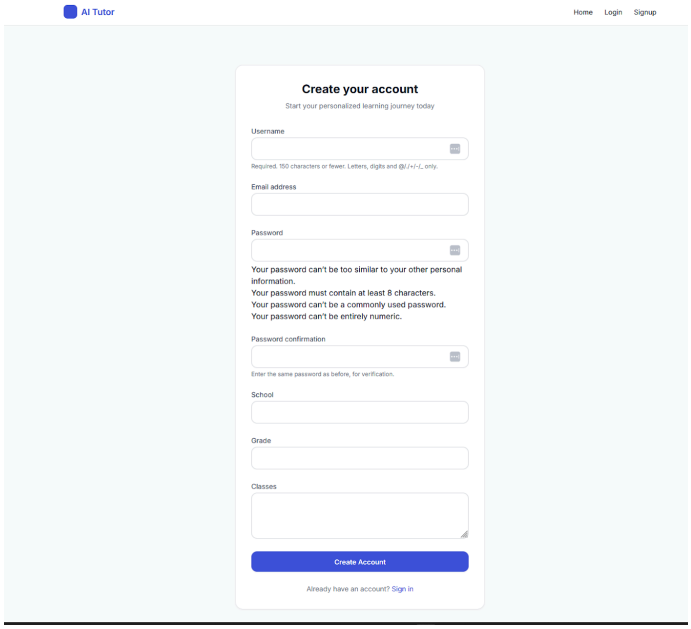
Students can be automatically signed up in bulk using a CSV exported from a database of student accounts. As of now, this uses an example CSV schema that may not be accurate to the CSV schema exported from other public school applications such as Skyward, but this will be changed in the future. Students can also sign up by themselves by connecting to the server’s IP address in a web browser and clicking the Sign-up button.

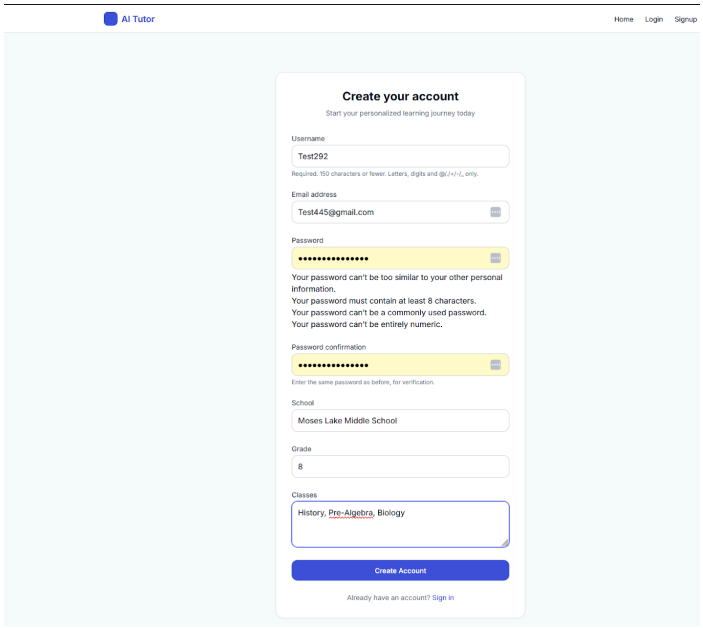
Students will input a username, a password, an email, and their list of subjects of study.

By default, there is an admin account with the username “admin” and the password “password1234”. This is meant to be changed immediately, before any students can be signed up. Admins can create mentor accounts and assign students to them.

The mentor and admin dashboards are also designed but not integrated into the main branch. The dashboard allows for mentors to view the students in their class and view their chat histories. The chat histories are stored on the server and obtained for the mentor and the student. The admin can also view the chat histories of any student in the system. Students can only view their own chat histories



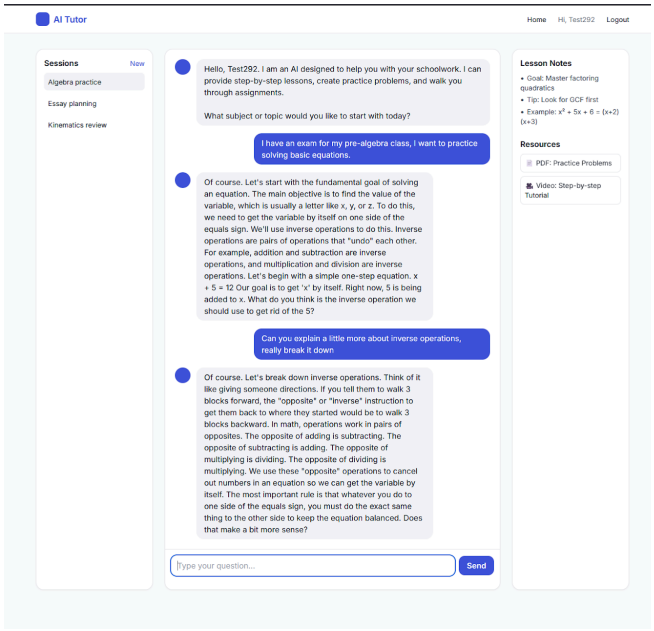




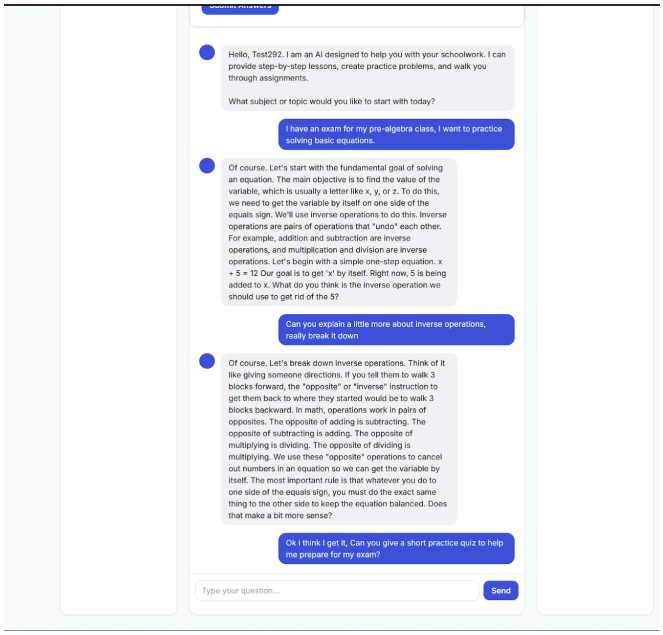
Project Title

X.III. Tutor Session Service

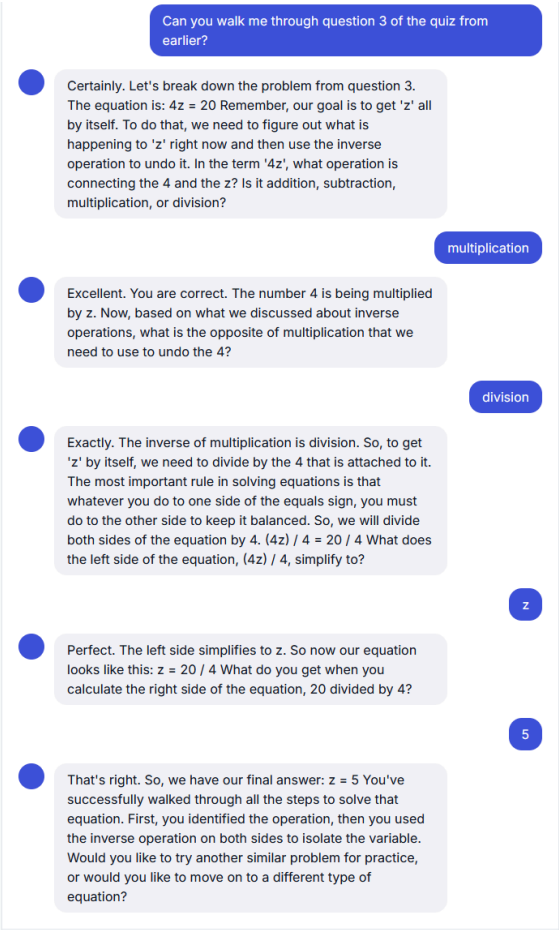
The chat bot will offer to help the student with the assigned subjects using its ingested RAG information. Students can ask questions for the chatbot, which will walk them through various topics. A specific system prompt is used to maintain professionalism and avoid inappropriate content or diverging from tutoring. You can start new sessions with the chatbot or view past sessions



.

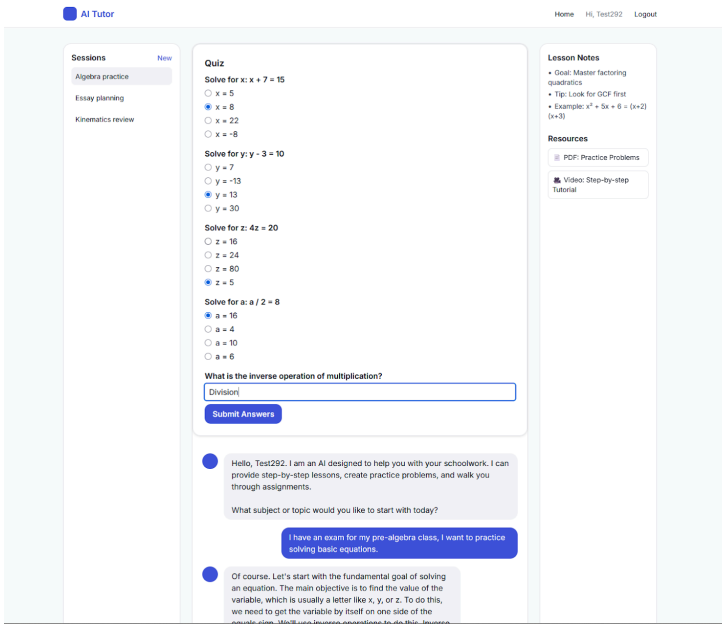


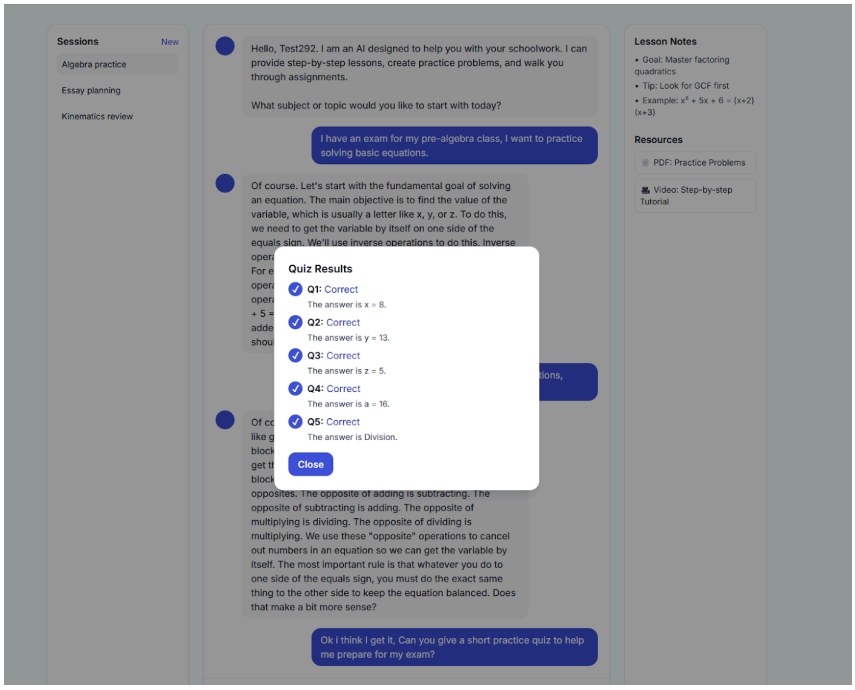
Project Title



X.IV. Quiz Service

The chat bot can be asked for a quiz, which it will return in a JSON format which is then used to assemble a quiz GUI. The quiz GUI features multiple choice options for the student to select. The system will then let the user know if they got a question wrong, and what the correct answer was. After the quiz, the user can ask clarifying questions about the quiz, since it stays in the chat history. Quizzes are fully implemented, but will sometimes not generate properly on the local models, and do not contribute to mastery tracking information.





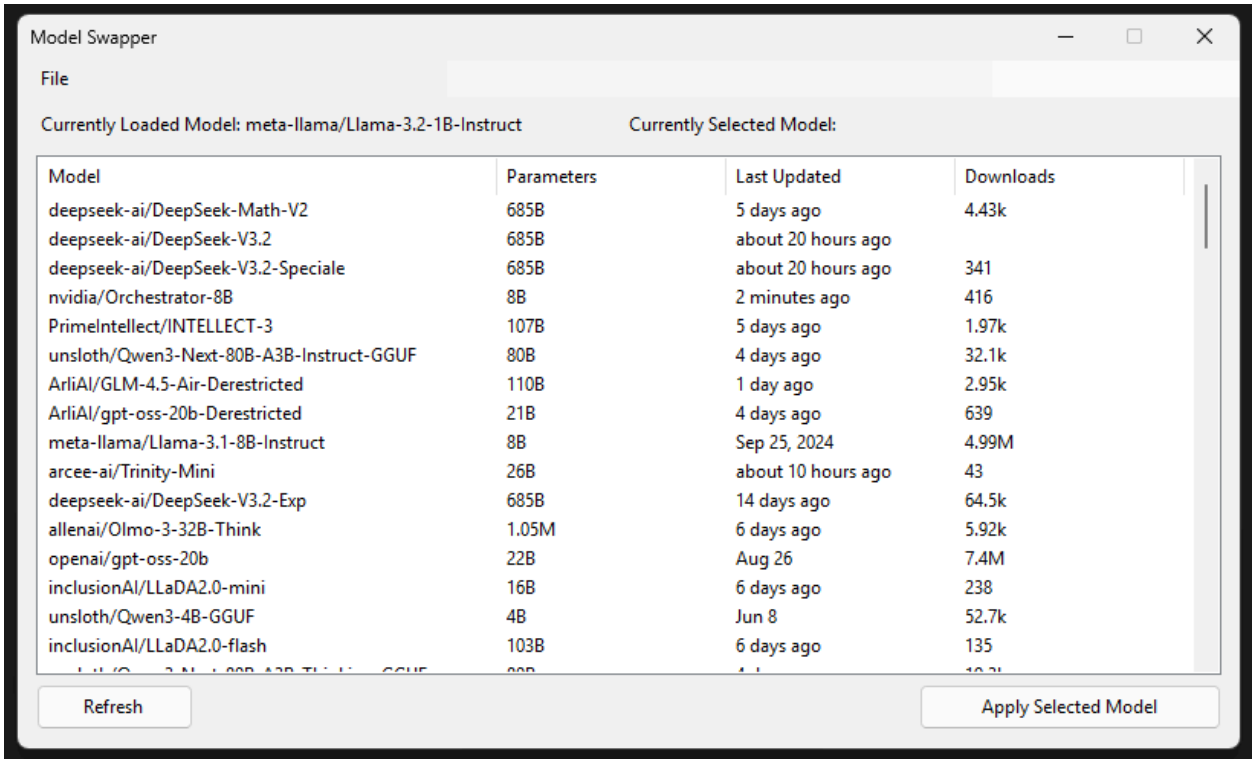
X.V. Content Moderation

The chatbot will not give inappropriate content. The system prompt restricts inappropriate content or chats and will try to stay relevant to the tutoring. Student chat histories are viewable by both mentors and admins, so if a mentor or an admin has suspicion that a student is not using the AI tutor appropriately, they can view the chat histories to confirm this. The mentors will also have access to their students' progress dashboards to make sure they are making progress.

X.VI. LLM Connector

Project Title

By default, the model is LLaMA-1B-Instruct and runs locally on the system, but there is also an option to switch to Google Gemini over the internet if the user provides an API key. There is also a helper tool for swapping the model with a click of a button rather than having to modify the .env file, which can be daunting for anybody who isn't an IT specialist.



Currently, RAG is implemented in the system, but it has not been given a proper front-end system to upload and remove files from the system. The files you wish to use in RAG must currently be put into a "curriculum" folder on the device that the system is running from.

X.VII. Progress and Mastery Service

The mastery and progress system is currently the last remaining piece to be implemented. The AI will be given instructions on how to score quizzes, which will directly integrate into this system. Students will be given a score for how well they do on quizzes on the subject. Quizzes, when scored, will change their overall score to be better or worse depending on how they did. Mentors are able to view the scores of all of the students in their class for all of their given subjects, and admins can view the scores of every student in the system for all of their given subjects.

X.VIII. ORM and Database Layer

User accounts are stored on the server along with their profiles, which includes their chat histories and any progress information.

XI. Alpha Prototype Demonstration

XI.1. Summary of Demonstration

During the alpha prototype demonstration, our team presented the foundational functionality of the AI Tutor system. The prototype showcased the core backend workflow, including the account system, bulk student onboarding through CSV imports, role-based access controls, and the early stages of the language model integration. We demonstrated how teachers can generate student accounts, how student profiles are automatically created and linked to Django's authentication system, and how the system logs and manages user interactions. Additionally, we reviewed the administrative console, the local language model setup, and the initial RAG-based ingestion pipeline used for processing PDFs and course materials. Our mentor was shown the current development environment, the testing workflow, and the key features now deployed such as the Web UI, local LLM integration, and content ingestion.

XI.2. Comments or Suggestions

Our mentor provided several important suggestions for improving the next phase of development. First, they recommended expanding the diagnostic and monitoring capabilities so teachers can more easily interpret student progress and receive timely intervention alerts. They also emphasized the importance of defining how adaptive difficulty should be calibrated and suggested testing various adaptation strategies before committing to a final model. Additionally, they encouraged us to ensure that privacy documentation aligns with FERPA expectations and that accessibility compliance is introduced early rather than at the end of development. Finally, the mentor suggested preparing for more robust system-level testing once the quiz engine and objective-tracking features are functional.

XI.3. Questions and Responses

Q: How does the system currently generate and manage student accounts?
A: The system uses a CSV ingestion workflow that automatically creates Django user accounts and associated StudentProfile entries, enabling quick classroom setup.

Q: Does the prototype support adaptive difficulty at this stage?
A: Not yet, only the foundational quiz structures are implemented. Adaptive difficulty logic will be introduced once question categorization and progression tracking are completed.

Q: How is the local LLM integrated into the prototype?
A: We demonstrated the current local model environment, which is configured but still being connected to the tutoring workflow. RAG processing and LLM query handling are functional but not yet connected to the UI experience.

Q: What components still require testing?
A: Unit tests for ingestion, role-based permissions, and student profiles are partially complete. Integration and system-level testing will begin once the quiz system and objectives module are stable.

XI.4. Planned Design Modifications

Based on mentor feedback and early testing, several design adjustments will be made. The adaptive difficulty system will be redesigned to incorporate clearer progression tiers and measurable mastery criteria defined by instructors. The teacher dashboard will be expanded to include diagnostic summaries and automated alerts for students who appear to be struggling. We also plan to refine the RAG ingestion pipeline for faster updates when new PDFs or materials are added, improve privacy safeguards in line with the client's requirements, and begin implementing accessibility accommodations across the Web UI. These changes will guide the next development sprints and shape the structure of our beta prototype.

XII. Future Work

In the second semester, our primary focus will shift toward rigorous testing of the system to ensure stability, accuracy, and consistent performance across all components of the AI Tutor. This includes validating the ingestion pipeline, verifying embedding quality, evaluating retrieval accuracy, and conducting end-to-end testing of the local language model’s responses. Through systematic testing, we aim to identify remaining issues, refine our design decisions, and ensure the system behaves reliably under real use.

We will also begin integrating the school server into our deployment workflow. Moving the system onto the university’s infrastructure will allow us to test real hosting conditions, verify performance outside of local machines, and ensure that all backend components including the local model, vector storage, and ingestion utilities function properly in a production-like environment. This step will also help us prepare the project for long-term maintainability beyond the capstone timeline.

In addition to testing and server integration, the team will finish implementing the remaining features planned for the AI Tutor. These include completing the Web UI interactions, finalizing the ingestion and retrieval tools, polishing the chat experience, and ensuring that all user-facing components are intuitive and fully functional. By the end of next semester, the goal is to deliver a complete, stable, and tested AI Tutor system ready for demonstration and final evaluation.

Glossary

LLM (Large Language Model) – A type of AI model trained on large text datasets that can generate human-like responses. Example: an 8B parameter model (8 billion parameters).

AI Tutor (LLM-based Tutor) - The language-model-driven reasoning engine that provides personalized guidance, feedback, and adaptive questioning.

RAG (Retrieval-Augmented Generation) – A method where an AI model retrieves relevant documents or content before generating a response, allowing the tutor to use course-specific materials.

Student-defined Learning Objectives – Goals and topics chosen by the student, which guide the AI tutor in generating personalized lessons and exercises.

RBAC (Role-Based Access Control) – A security method that assigns system permissions based on user roles (e.g., Student, Teacher, Admin).

FERPA (Family Educational Rights and Privacy Act) – U.S. law that protects the privacy of student education records.

COPPA (Children's Online Privacy Protection Act) – U.S. law that restricts the collection of personal data from children under 13 without parental consent.

SSO (Single Sign-On) – A login method that allows users to access multiple systems with one set of credentials.

SAML (Security Assertion Markup Language) – A standard protocol for exchanging authentication and authorization data in SSO systems.

OAuth2 – An open standard for access delegation, allowing secure authorization for applications.

Encryption in Transit – Securing data while it is being sent between systems.

Encryption at Rest – Securing data when it is stored on servers or devices).

Mentor - Serves the role of the "teacher" in traditional schooling. The mentor will track students' progress with their AI tutor using the mastery tracking systems. If a student is struggling too badly in a certain topic, the mentor may be alerted to their struggles so that they can provide extra assistance.

Bulk Account Creation - A mechanism that allows administrators to upload a CSV file to create multiple student accounts automatically.

Continuous Integration (CI) - An automated process using tools such as GitHub Actions that runs unit and integration tests whenever new code is pushed or merged.

Django Backend - The primary server framework used to implement authentication, routing, storage, dashboards, and API endpoints for the AI Tutor system.

Model Swapper - A helper tool included in the project that makes selecting a language model easier for a school IT professional. Rather than having to edit a config file, they can press two buttons to swap the model.

Non-Deterministic Output - Behavior where the same prompt may produce different LLM responses, making traditional unit assertions impossible.

Role-Based Access Control (RBAC) - A security model that restricts system capabilities based on predefined user roles: Student, Teacher, and Administrator.

References

[1] D. Sleeman and J. S. Brown, Intelligent Tutoring Systems. London: Academic Press, 1982.

[2] Khan Academy, “Khanmigo: Khan Academy’s AI Tutor,” 2023. Available: https://www.khanacademy.org/khan-labs

[3] MagicSchool, “MagicSchool.ai: AI Tools for Educators,” 2023. Available: https://www.magicschool.ai

[4] UNESCO, “AI and the Future of Education: Policy Brief,” 2021. Available: https://unesdoc.unesco.org

[5] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” Meta AI, 2023.

[6] Mistral AI, “Introducing Mistral 7B,” 2023. Available: https://mistral.ai

[7] “Inside San Francisco’s new AI school: is this the future of US education?” by Robin Buller, The Guardian, 2025. https://www.theguardian.com/technology/2025/oct/18/san-francisco-ai-alpha-school-tech